Verification and Validation Report: Physiotherapy Companion

Team 11, technically functional
Matthew Huynh
Cieran Diebolt
Vaisnavi Shanthamoorthy
Maham Siddiqui
Eman Ashraf

March 9, 2026

1 Revision History

Date	Version	Notes
March 9th, 2026	1.0	Created inital version of VnV Report
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

symbol	description
TC	Test Case
UT	Usability Test
PR	Performance Test
RFT	Reliability and Fault-Tolerance Test
SR	Security Test
MS	Maintainability and Support Test
FR	Functional Requirement
SR-AR	Security Access Requirement

Contents

List of Tables

List of Figures

This document ...

3 Functional Requirements Evaluation

1. **test-TC1 : Physiotherapist Registration (Valid License)**

   Initial State: App launched; registration screen accessible; Firebase Auth and Firestore active.

   Input: `registerPhysio()` called with a valid license format and an active license in `validLicenses`.

   Output: Auth user created. Firestore `users/{uid}` written with `role="physio"`, `verified=true`, and `createdAt`. All assertions passed.

   Result: Pass

2. **test-TC2 : Physiotherapist Registration (Invalid License Format)**

   Initial State: App launched; registration screen accessible.

   Input: `registerPhysio()` called with an invalid license format (missing dash or incorrect digits).

   Output: Error "Invalid license format" thrown. No Auth user or Firestore writes created.

   Result: Pass

3. **test-TC3 : Physiotherapist Registration (Unverified License)**

   Initial State: App launched; registration screen accessible.

   Input: `registerPhysio()` called with valid format but license not found or `active=false` in `validLicenses`.

   Output: Error "License not verified" thrown. No Auth user or Firestore writes created.

   Result: Pass

4. **test-TC4 : Patient Registration (Valid Invite Code)**

   Initial State: App launched; `inviteCodes` collection seeded with an active, unused code containing a `physioId`.

Input: `registerPatient()` called with a valid invite code.

Output: Auth user created. Firestore `users/{uid}` written with `role="patient"` and stored `physioId`. Invite code updated with `used=true` and `usedBy={uid}`.

Result: Pass

5. **test-TC5 : Patient Registration (Non-Existent Invite Code)**

   Initial State: App launched; `inviteCodes` collection does not contain the submitted code.

   Input: `registerPatient()` called with a code that does not exist in `inviteCodes`.

   Output: Error "Invalid invite code" thrown. No Auth user or Firestore writes created.

   Result: Pass

6. **test-TC6 : Patient Registration (Inactive Invite Code)**

   Initial State: App launched; `inviteCodes` contains a matching code with `active=false`.

   Input: `registerPatient()` called with an inactive invite code.

   Output: Error "Invite code inactive" thrown. No Auth user or Firestore writes created.

   Result: Pass

7. **test-TC7 : Patient Registration (Already Used Invite Code)**

   Initial State: App launched; `inviteCodes` contains a matching code with `used=true`.

   Input: `registerPatient()` called with an already used invite code.

   Output: Error "Invite code already used" thrown. No Auth user or Firestore writes created.

   Result: Pass

8. **test-TC8 : Patient Registration (Invite Code Missing Physio Link)**

   Initial State: App launched; `inviteCodes` contains a matching code with no `physioId` field.

   Input: `registerPatient()` called with a code missing a `physioId`.

   Output: Error "Invite code missing physio link" thrown. No Auth user or Firestore writes created.

   Result: Pass

9. **test-TC9 : User Login (Valid Physiotherapist Credentials)**

   Initial State: Login screen available; Firestore contains a profile with `role="physio"`.

   Input: `loginUser()` called with valid PT credentials.

   Output: Login returns object with `uid` and profile data including `role="physio"`. No errors thrown.

   Result: Pass

10. **test-TC10 : User Login (Valid Patient Credentials)**

    Initial State: Login screen available; Firestore contains a profile with `role="patient"`.

    Input: `loginUser()` called with valid patient credentials.

    Output: Login returns object with `uid` and profile data including `role="patient"`. No errors thrown.

    Result: Pass

11. **test-TC11 : User Login (Missing Firestore Profile)**

    Initial State: Login screen available; valid Firebase Auth credentials exist but no Firestore profile.

    Input: `loginUser()` called with valid credentials.

    Output: Error "User profile missing in Firestore" thrown. Login rejected.

    Result: Pass

12. **test-TC12 : User Logout**

    Initial State: User session active.

    Input: `logoutUser()` called.

    Output: Auth sign-out called once and completes without error.

    Result: Pass

13. **test-TC13 : Get User Data**

    Initial State: Firestore populated with test user documents.

    Input: `getUserData()` called with a valid uid, a non-existent uid, and an empty uid.

    Output: Valid uid returns `UserData`. Non-existent uid returns null. Empty uid throws an error.

    Result: Pass

14. **test-TC14 : User Lookup by Email**

    Initial State: Firestore populated with test user documents.

    Input: `getUserByEmail()` called with a registered email and an unregistered email.

    Output: Registered email returns user object and true. Unregistered email returns null and false.

    Result: Pass

15. **test-TC15 : Update User Data**

    Initial State: Firestore populated with test user documents.

    Input: `updateUser()` called with a valid update payload.

    Output: Update returns true. Changes reflected in Firestore.

    Result: Pass

16. **test-TC16 : Delete User**

    Initial State: Firestore populated with test user documents.

    Input: `deleteUser()` called with a valid uid, a valid email, and a non-existent case.

Output: Valid cases return true. Non-existent case returns false.

Result: Pass

17. **test-TC17 : User Queries**

    Initial State: Firestore populated with linked PT and patient records.

    Input: `getPatientsByPhysioId()`, `searchByName()`, and `getAllUsers()` called.

    Output: Each function returns correctly filtered arrays. Errors return empty arrays.

    Result: Pass

18. **test-TC18 : User Info Lookup**

    Initial State: Firestore populated with linked PT and patient records.

    Input: `getUserdbInfo()` called with name only and with name and birthday filter.

    Output: Returns categorized patients and physiotherapists correctly. Birthday filter works as expected.

    Result: Pass

19. **test-TC19 : PT Account Delete**

    Initial State: Firestore populated with a valid physiotherapist and linked patient.

    Input: `deletePTAccount()` called with a valid physio, a non-physio user, and a non-existent patient.

    Output: Valid case deletes patient successfully. Non-physio and non-existent cases throw appropriate errors.

    Result: Pass

20. **test-TC20 : User Authentication Validation**

    Initial State: Firestore populated with test user credentials.

    Input: `usernamePwMatch()` and `authenticateUser()` called with valid credentials, an invalid password, and a non-existent user.

Output: Valid credentials return true and user object. Invalid password throws an error. Non-existent user returns null.

Result: Pass

21. **test-TC21 : System Initialization**

    Initial State: System not yet initialized.

    Input: `initializeSystem()` called.

    Output: Success message logged to console. No errors thrown.

    Result: Pass

22. **test-TC22 : Credential Validation**

    Initial State: User account exists in Firestore.

    Input: `validateCredentials()` called with a correct password and an incorrect password.

    Output: Correct password returns true. Incorrect password returns false.

    Result: Pass

23. **test-UT-4 : Account Creation (Usability)**

    Initial State: App installed on participant's device; invite code and PT license number provided.

    Input: Participant launched the app, selected their role, and completed account creation.

    Output: All participants created accounts successfully without assistance. PT license and invite code fields completed without difficulty. Minor font size feedback noted and resolved.

    Result: Pass

24. **test-UT-5 : Exercise Recording Initiation**

    Initial State: Patient logged in on mobile device; camera permission granted; backend server running.

    Input: Participant navigated to the Record screen and pressed Start Recording.

Output: Camera activated and pose detection began. On-screen prompts instructed participant to enter the detectable frame area. Navigation and button labels reported as clear.

Result: Pass

25. **test-UT-6 : Real-Time Pose Detection and Form Feedback**

    Initial State: Patient positioned within camera frame; pose detection active; backend returning joint angle and ROM metrics.

    Input: Participant performed knee extension repetitions.

    Output: App detected relevant body landmarks and displayed appropriate form feedback overlays. Rep count incremented correctly. Detection accuracy was reported as satisfactory.

    Result: Pass

26. **test-UT-7 : Exercise Page (Instructions)**

    Initial State: Patient logged in; Exercises tab accessible.

    Input: Participant navigated to the Exercises page and reviewed listed exercises.

    Output: Exercises displayed with names, target muscle groups, set/rep counts, and step-by-step instructions. Instructions reported as clear and easy to follow.

    Result: Pass

27. **test-UT-8 : Progress Screen (Session History)**

    Initial State: Patient logged in with at least one completed session stored in Firestore.

    Input: Participant navigated to the Progress tab.

    Output: Session history displayed with ROM and rep count data. Layout was readable and logically arranged. Participants suggested adding sets and pain level as future enhancements.

    Result: Pass

28. **test-UT-9 : PT Dashboard (Patient List and Activity)**

    Initial State: Physiotherapist logged in; at least one linked patient with recorded sessions.

    Input: PT participant reviewed the home dashboard and selected a patient to view their activity history.

    Output: Dashboard displayed linked patients and activity history. Patient detail view showed session history and ROM chart. Layout reported as clear and sufficient for clinical use.

    Result: Pass

29. **test-UT-10 : PT Feedback Submission**

    Initial State: Physiotherapist logged in; patient session detail page accessible.

    Input: PT participant selected a session entry and submitted a written comment.

    Output: Feedback saved to Firestore and associated with the session. Comment visible from the patient's session view. PT participants rated the feature as clinically useful.

    Result: Pass

4 Nonfunctional Requirements Evaluation

4.1 Usability

1. **UT-1 : Navigation Clarity**

   Initial State: App installed on a mobile device; user logged in.

   Input: User was asked to find and open the exercise screen, start a recording session, and return to the dashboard without assistance.

   Output: User successfully navigated through the required screens without confusion. Feedback indicated labels and buttons were clear.

   Result: Pass

2. **UT-2 : Instruction Clarity During Exercise**

   Initial State: User is on the record exercise screen with camera access enabled.

   Input: User initiated an exercise session. The application displayed live on-screen feedback instructing the user of any adjustments.

   Output: User reported that instructions were clear and prompts matched the required actions. Minor wording improvements were noted and updated.

   Result: Pass

3. **UT-3 : Survey Questionnaire – ui_testing**

   Initial State: App accessible to testers on mobile devices.

   Input: Five users completed a short usability questionnaire addressing ease of use, clarity and navigation.

   Output: Average rating across categories was $\geq 4/5$. Minor UI refinements (font size and button spacing) were applied based on feedback.

   Result: Pass

4.2 Performance

1. **PR-1 : Exercise Interface Response Time – ex_int_speed**

   Initial State: User logged in on a mobile device; backend server running.

   Input: User started recording from the record screen. Screen recording timestamps were used to measure the time from tapping "Start" to the first visible feedback/result. There were five trials conducted.

   Output: Measured response times were 1.87s, 1.92s, 1.79s, 1.95s, and 1.84s. The average end-to-end response time was 1.87 seconds ($\leq$ 2 seconds).

   Result: Pass

2. **PR-2 : Repeated Use Performance**

   Initial State: Application running normally on mobile device.

Input: User completed 10 consecutive "Start → Record Exercise → Stop" cycles.

Output: There was no significant slowdown observed across the 10 trials. Additionally, the response times were all within acceptable range and no crashes took place.

Result: Pass

4.3 Reliability and Fault-Tolerance

1. **RFT-1 : Backend Failure Handling – fail_recovery**

   Initial State: User actively using the application while connected to backend.

   Input: Backend server was manually terminated during active analysis.

   Output: Application displayed a network error message and remained responsive (no crash). After backend restart, the user was able to retry and continue normal operation.

   Result: Pass

2. **RFT-2 : Network Interruption During Upload – Interrupted_vid**

   Initial State: User uploading recorded exercise video.

   Input: WiFi connection was disabled mid-upload and restored after approximately 10 seconds.

   Output: Application detected upload failure and displayed a retry option. User retried successfully and uploaded video was verified to be intact.

   Result: Pass

4.4 Security

1. **SR-1 : Unauthorized Access to Protected Endpoint**

   Initial State: Backend server running with authentication enabled.

   Input: A request was sent to a protected endpoint without authentication credentials.

Output: Server returned HTTP 401 Unauthorized. Additionally, no protected data was returned.

Result: Pass

2. **SR-2 : Invalid Credentials Login**

   Initial State: Application login screen available.

   Input: Login attempted using an email not registered in the database or incorrect password.

   Output: System denied login and displayed an appropriate error message without exposing any system details.

   Result: Pass

4.5 Maintainability and Support

1. **MS-1 : Reproducible Setup**

   Initial State: Fresh machine with no prior installation.

   Input: Project repository was cloned and installed using instructions provided in README (the frontend and backend setup).

   Output: Application ran successfully without missing dependencies. All documented setup steps were sufficient to successfully reproduce the system.

   Result: Pass

2. **MS-2 : Code Review and Quality Assurance**

   Initial State: Revision 0 completed.

   Input: Numerous peer code reviews were conducted through pull requests and issue tracking. The unit tests were executed to validate core functionality, including the metric calculations and functional logic. The UI components were reviewed for usability, clarity of instructions, and error handling. Moreover, the identified issues were documented and resolved.

Output: The core functions passed unit testing. The UI refinements were implemented based on peer feedback. The code structure maintained clear separation between modules and frontend components. Overall, there were no major functional issues identified during review.

Result: Pass

5 Comparison to Existing Implementation

This section will not be appropriate for every project.

6 Unit Testing

The following section outlines the unit tests created for all of the modules.

6.1 Authentication

6.1.1 Account Creation Module

The following tests in Table 1 verify the functionality of the account creation module for both physiotherapists and patients, including validation checks and successful user creation workflows.

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC1	FR1.1, FR1.2	Register physiotherapist with valid license format and a license that exists in `validLicenses` with `active=true`.	Auth user is created. Firestore `users/{uid}` document is written with `role="physio"`, trimmed name, lowercased email, uppercased license number, `verified=true`, and `createdAt`.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC2	FR1.1, FR1.2	Register physiotherapist with an invalid license format (e.g., a missing dash or incorrect digits).	Error `"Invalid license format (example: ON-123456)."` is thrown. No Auth user is created. No Firestore writes occur.	All assertions pass.	Pass
TC3	FR1.1, FR1.2	Register physiotherapist with valid format but license is not found in `validLicenses` or `active != true`.	Error `"License not verified."` is thrown. No Auth user is created. No Firestore writes occur.	All assertions pass.	Pass
TC4	FR1.1, FR1.3, FR2, FR3	Register patient with invite code that exists in `inviteCodes`, is `active=true`, `used=false`, and includes a `physioId`.	Auth user is created. Firestore `users/{uid}` document is written with `role="patient"`, trimmed name, lowercased email, stored `physioId`, normalized invite code, and `createdAt`. Invite is updated with `used=true` and `usedBy={uid}`.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC5	FR1.3, FR3	Register patient with an invite code that does not exist in `inviteCodes`.	Error `"Invalid invite code."` is thrown. No Auth user is created. No Firestore writes/updates occur.	All assertions pass.	Pass
TC6	FR1.3, FR3	Register patient with an invite code that exists but `active=false`.	Error `"Invite code inactive."` is thrown. No Auth user is created. No Firestore writes/updates occur.	All assertions pass.	Pass
TC7	FR1.3, FR2	Register patient with an invite code that exists but `used=true`.	Error `"Invite code already used."` is thrown. No Auth user is created. No Firestore writes/updates occur.	All assertions pass.	Pass
TC8	FR1.3, FR2	Register patient with an invite code that exists but missing `physioId`.	Error `"Invite code missing physio link."` is thrown. No Auth user is created. No Firestore writes/updates occur.	All assertions pass.	Pass

Table 1: Account Creation Module Test Cases

6.1.2 Valid User Module

The following tests in Table 2 verify the functionality of the valid user module.

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC9	FR1, FR1.1, SR-AR1	Login with valid credentials for a physiotherapist whose Firestore profile `users/{uid}` exists and contains `role="physio"`.	Login returns an object containing the `uid` and stored profile data (including role and user fields). No errors occur.	All assertions pass.	Pass
TC10	FR1, FR1.1, SR-AR1	Login with valid credentials for a patient whose Firestore profile `users/{uid}` exists and contains `role="patient"`.	Login returns an object containing the `uid` and stored profile data (including role and user fields). No errors occur.	All assertions pass.	Pass
TC11	FR1, FR1.1, SR-AR1	Login with valid credentials where the Firestore profile `users/{uid}` does not exist.	Error `"User profile missing in Firestore."` is thrown and login is rejected.	All assertions pass.	Pass
TC12		Logout after a user session is active.	The Auth sign-out operation is called once and completes without error.	All assertions pass.	Pass

Table 2: Valid User Module Test Cases

6.2 User Account module

The tests below outline the User Account module functionality. Note that many new functions were added for this iteration. Currently, it serves as an interface between the database and the various other components. Additionally, there may be some overlap between the Account Creation module and Valid User module. While the modules themselves perform a few differ-

ent functions, they are inextricably linked due to their handling of the same entity. For example, the Valid User module consists of tests the ensure successful Login and the User Account module test cases extend that to testing for invalid and non-existent users.

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
USER MANAGEMENT FUNCTIONS					
TC13	FR23, FR21	Get User Data: Test with valid `uid`, non-existent `uid`, and empty `uid`.	Valid `uid` returns `UserData`; non-existent returns null; empty throws error.	All assertions pass.	Pass
TC14	FR21	User Lookup: Test get user by email and email existence check with registered and unregistered emails.	Registered email returns user and true; unregistered returns null and false.	All assertions pass.	Pass
TC15	FR2, FR6, FR22	Update User: Test update user data.	Valid update returns true.	All assertions pass.	Pass
TC16	FR4	Delete User: Test delete by `uid` and by email with valid and non-existent cases.	Valid cases return true; non-existent return false.	All assertions pass.	Pass
QUERY FUNCTIONS					
TC17	FR18	User Queries: Test get patients by physioId, search by name, and get all users.	Returns correctly filtered arrays; errors return empty arrays.	All assertions pass.	Pass
TC18	FR21, FR25	User Info Lookup: Test getUserdbInfo with name only and with name+birthday filter.	Returns categorized patients/physios correctly; filtering works as expected.	All assertions pass.	Pass
SPECIALIZED FUNCTIONS					

Continued on next page

Continued from previous page

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC19	FR2, FR4, FR22	PT_Account_Delete: Test with valid physio, non-physio user, and non-existent patient.	Valid case deletes patient; non-physio throws error; non-existent throws error.	All assertions pass.	Pass
TC20	SR-AR1, SR-PR1	Test usernamePwMatch and authenticateUser with valid credentials, invalid password, and non-existent user.	Valid returns true/user; invalid throws appropriate errors; non-existent returns null.	All assertions pass.	Pass
TC21	SR-AR1, SR-AR2	System Initialization: Test initializeSystem.	Logs success message to console.	All assertions pass.	Pass
		VALIDATION HELPERS			
TC22	FR1, SR-AR1	Test validateCredentials with correct and incorrect passwords.	Correct returns true; incorrect returns false.	All assertions pass.	Pass

Table 3: User Account Service Test Cases

6.3 Live Feedback Module

The following tests in Table 4 verify the functionality of the live feedback module, including real-time corrective message generation, safety warning triggers, rep counting, and physiotherapist comment submission and retrieval.

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC23	FR13, FR14	Submit joint angle measurements within the acceptable range for a knee extension rep.	No corrective feedback message is generated. Rep count increments by one. Positive reinforcement message returned.	All assertions pass.	Pass
TC24	FR13, FR14	Submit joint angle measurements below the minimum threshold for knee extension (insufficient range of motion).	Corrective feedback message is generated (e.g., `"Extend your knee further to complete the rep."`). Rep is not counted.	All assertions pass.	Pass
TC25	FR13, FR14	Submit joint angle measurements above the maximum safe threshold, simulating an overextension.	Corrective feedback message is generated (e.g., `"Reduce the range of your movement."`). Rep is not counted.	All assertions pass.	Pass
TC26	FR16, PR-SCR1	Submit joint angle measurements that exceed the defined dangerous movement threshold.	Safety warning is triggered (e.g., `"Stop immediately | dangerous movement detected."`). Exercise session is halted.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC27	FR14, PR-SL2	Submit a sequence of 5 consecutive valid reps and verify feedback generation latency for each.	Feedback is returned for each rep within 2 seconds. Rep count reflects 5 completed reps. No degradation in response time across reps.	All assertions pass.	Pass
TC28	FR14	Invoke the feedback generator with no pose landmarks detected (user out of frame).	Feedback message returned is `"Please step into the camera frame."` No rep is counted. No safety warning triggered.	All assertions pass.	Pass
TC29	FR20, FR25	Physiotherapist submits a written comment on a completed patient session.	Comment is saved to Firestore and associated with the correct session document. No errors thrown.	All assertions pass.	Pass
TC30	FR19, FR25	Patient retrieves physiotherapist feedback for a session that has an associated PT comment.	Correct comment text is returned and associated with the matching session. Non-existent session returns null.	All assertions pass.	Pass

Table 4: Live Feedback Module Test Cases

6.4 Video Capture and Review module

The tests below outline the Video Capture and Review functionality. Some of the interactions with following tests as well as other modules may touch on the video capture and review module, but the tests below are focused on the core functionalities of the module itself.

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC23	FR12	Call `get_pose_connections(legs_only=False)` and verify against `POSE_CONNECTIONS`.	`POSE_CONNECTIONS` and `LEG_CONNECTIONS` have length greater than 0. `LEG_CONNECTIONS` length is less than `POSE_CONNECTIONS`, and the set of connections from `get_pose_connections(False)` equals the set of `POSE_CONNECTIONS`.	All assertions pass.	Pass
TC24	FR12	Call `get_pose_connections(legs_only=True)` and verify against `LEG_CONNECTIONS`.	The connections returned equal the set of `LEG_CONNECTIONS`.	All assertions pass.	Pass
TC25	FR12	Call `extract_landmarks()` with a valid pose result containing 33 landmarks.	Returns a list of 33 landmarks. Each landmark has x, y, z values that are close to the expected i/33 values.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC26	FR26	Call `extract_landmarks()` with None and with an empty result (pose_landmarks=None).	Returns None for both.	All assertions pass.	Pass
TC27	FR7	Initialize PoseCamera with mocked VideoCapture and mocked PoseLandmarker.	Camera initializes successfully. `display_size` is set to (640, 480). `cap.set` is called to configure frame dimensions.	All assertions pass.	Pass
TC28	FR7	Call `get_frame()` with mocked camera that returns a numpy array frame.	Returns (True, frame) where frame is a numpy array. Frame is resized to display dimensions.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC29	FR12	Call `process_pose()` with a mocked frame.	Returns a mock result. The pose detector's `detect` method is called once with the converted RGB image.	All assertions pass.	Pass
TC30	FR14	Call `draw_landmarks()` with a mock result and a numpy array frame.	Returns the frame. OpenCV's `line` and `circle` functions are called to draw the pose landmarks.	All assertions pass.	Pass

Table 5: Video Capture and Review Module Test Cases

6.5 Computer Vision and Form Analysis Module

The following tests in Table 6 verify the functionality of the computer vision and form analysis module by ensuring that joint angles are accurately calculated, knee angles are correctly extracted from pose landmarks, the analyzer calibrates to the user's range of motion, repetitions are counted properly, and summary metrics are generated as expected.

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC31	FR12	Call `angle_deg()` with points forming a 90-degree angle: a=(0,1), b=(1,1), c=(1,0).	Returns 90.0 degrees (within 0.01 tolerance).	All assertions pass.	Pass
TC32	FR12	Call `angle_deg()` with points forming a 45-degree angle: a=(0,1), b=(1,1), c=(0,0).	Returns 45.0 degrees (within 0.01 tolerance).	All assertions pass.	Pass
TC33	FR12	Call `angle_deg()` with points forming a 180-degree straight line: a=(0,1), b=(1,1), c=(2,1).	Returns 180.0 degrees (within 0.01 tolerance).	All assertions pass.	Pass
TC34	FR26	Call `angle_deg()` with None values for each parameter position: (None, (1,1), (1,0)), ((0,1), None, (1,0)), ((0,1), (1,1), None).	Returns None for all three cases.	All assertions pass.	Pass
TC35	FR12	Call `knee_angle_from_result()` with a valid mock result containing right leg landmarks (hip=24, knee=26, ankle=28) forming a 90-degree angle.	Returns a valid angle value between 0 and 180 degrees.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC36	FR12	Call `knee_angle_from_result()` with a valid mock result containing left leg landmarks (hip=23, knee=25, ankle=27) forming a 90-degree angle.	Returns a valid angle value between 0 and 180 degrees.	All assertions pass.	Pass
TC37	FR26	Call `knee_angle_from_result()` with None instead of a result object.	Returns None.	All assertions pass.	Pass
TC38	FR26	Call `knee_angle_from_result()` with a result object that has `pose_landmarks=None`.	Returns None.	All assertions pass.	Pass
TC39	FR26	Modify the mock result by setting the right knee landmark (index 26) to None, then call `knee_angle_from_result()` with side="RIGHT".	Returns None due to missing landmark.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC40	FR26	Modify the mock result by setting the left knee landmark (index 25) to None, then call `knee_angle_from_result()` with side="LEFT".	Returns None due to missing landmark.	All assertions pass.	Pass
TC41	FR26	Set the mock result's `pose_landmarks` to an empty list and call `knee_angle_from_result()`.	Returns None due to empty landmarks.	All assertions pass.	Pass
TC42	FR12	Initialize Analyzer with default values.	`started=False`, `min_angle=0.0`, `max_angle=0.0`, `is_calibrating=False`, `cal_start_ts=None`, `cal_duration_s=10.0`, `cal_min=None`, `cal_max=None`, `rep_count=0`, `current_rep=None`, `rep_durations=[]`, `rep_state=""`.	All assertions pass.	Pass
TC43	FR8, FR27	Call `start_calibration(duration_s=5.0)`	`is_calibrating=True`, `cal_duration_s=5.0`, `cal_start_ts` is set, `cal_min=None`, `cal_max=None`.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC44	FR8, FR27	During calibration, update with angles [30, 45, 60, 50, 70, 20].	`cal_min` updates to min value (20), `cal_max` updates to max value (70).	All assertions pass.	Pass
TC45	FR8, FR27	During calibration with duration 0.1s, update with angle 50 at t=100.05s (within duration), then angle 60 at t=100.16s (past duration).	First update: `is_calibrating` remains True. Second update: `is_calibrating` becomes False, `started` becomes True, `min_angle=50`, `max_angle=60`.	All assertions pass.	Pass
TC46	FR8, FR26	During calibration, call `_calibrate_with_angle(None)`.	Calibration ignores None values. `cal_min` and `cal_max` remain None.	All assertions pass.	Pass
TC47	FR8, FR16	Set `min_angle=30.0`, `max_angle=70.0`, `started=True`	`rep_state` is either "Extension" or empty.	All assertions pass.	Pass
TC48	FR8, FR14	With rep state "Extension", update with angle 35.	Rep state transitions to "Flexion". `current_rep` timestamp is updated.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC49	FR8, FR11	With rep state "Flexion" and `current_rep=100.0`, update with angle 68 at t=101.5s (1.5s later).	Rep state transitions to "Extension". Rep count increments to 1. Rep duration of 1.5s is recorded.	All assertions pass.	Pass
TC50	FR8, FR11	With rep state "Flexion" but `current_rep=None`, update with angle 68.	Rep state transitions to "Extension" but no rep is counted. Rep count remains 0.	All assertions pass.	Pass
TC51	FR8, FR12	Call `update(25)` and `update(75)` and verify min/max expansion via `_rep_update`.	`min_angle` updates to 25, `max_angle` updates to 75.	All assertions pass.	Pass
TC52	FR8, FR27	During calibration mode, call `update(45.0)`.	`_calibrate_with_angle` is called with 45.0.	All assertions pass.	Pass
TC53	FR8, FR12	Before `started` is set, call `update(45.0)`.	`min_angle` and `max_angle` are both set to 45.0. `started` becomes True.	All assertions pass.	Pass
TC54	FR8, FR27	With `started=True`, `min_angle=30.0`, `max_angle=70.0`, call `update(20.0)`.	`min_angle` updates to 20.0. `max_angle` remains 70.0.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC55	FR8, FR27	With `started=True`, `min_angle=30.0`, `max_angle=70.0`, call `update(80.0)`.	`max_angle` updates to 80.0. `min_angle` remains 30.0.	All assertions pass.	Pass
TC56	FR8, FR26	With `started=True`, call `update(None)`.	Values remain unchanged: `min_angle=30.0`, `max_angle=70.0`.	All assertions pass.	Pass
TC57	FR11, FR17	With `min_angle=30.0`, `max_angle=70.0`, `rep_count=5`, `rep_state="Extension"`, `rep_durations=[1.2, 1.3, 1.1, 1.4, 1.2]`, call `summary()`.	Summary contains all expected keys: `min_degree`, `max_degree`, `rom_degree`, `calibrating`, `cal_time_left`, `rep_count`, `current_rep_duration`, `avg_rep_duration`, `rep_state`.	All assertions pass.	Pass
TC58	FR11, FR17	With `min_angle=30.123`, `max_angle=70.456`, `rep_count=5`, `rep_state="Extension"`, `rep_durations=[1.2, 1.3, 1.1, 1.4, 1.2]`, call `summary()`.	`min_degree` rounds to 30.1, `max_degree` rounds to 70.5, `rom_degree` calculates to 40.3.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC59	FR8, FR27	During calibration with `cal_start_ts=100.0`, `cal_duration_s=10.0`, at current time 105.0, call `summary()`.	`calibrating=True`, `cal_time_left=5.0`.	All assertions pass.	Pass
TC60	FR11, FR17	When not calibrating, call `summary()`.	`calibrating=False`, `cal_time_left=0.0`.	All assertions pass.	Pass
TC61	FR11, FR17	With `rep_durations=[]` (no reps completed), call `summary()`.	`avg_rep_duration=0`, `current_rep_duration=0`.	All assertions pass.	Pass
TC62	FR11, FR8	With `current_rep=100.0` and `rep_durations=[1.2, 1.3]`, at current time 101.5, call `summary()`.	`current_rep_duration=1.5`, `avg_rep_duration=1.25`.	All assertions pass.	Pass
TC63	FR8, FR12	With `min_angle=50.0`, `max_angle=50.0`, `started=True`, call `_rep_update(50.0)`.	Function handles gracefully without division by zero errors.	All assertions pass.	Pass
TC64	FR12, FR26	With `started=True`, `min_angle=-10.0`, `max_angle=30.0`, update with angle -20.0 then 40.0.	`min_angle` expands to -20.0, `max_angle` expands to 40.0.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC65	FR12, FR26	With `started=True`, `min_angle=0.0`, `max_angle=180.0`, update with angle 200.0.	`max_angle` expands to 200.0.	All assertions pass.	Pass
TC66	FR8	With `min_angle=50.0`, `max_angle=50.0`, `started=True`, `rep_state="Extension"`, call `_rep_update(50.0)`.	No state change occurs. Rep state remains "Extension". Rep count remains 0.	All assertions pass.	Pass

Table 6: Computer Vision and Form Analysis Module Test Cases

6.6 Performance and Latency Module

The following tests in Table 7 verify the functionality of the performance and latency module by ensuring that the backend services initialize correctly, API endpoints respond appropriately to valid and invalid requests, frame processing handles mirroring and landmark detection correctly, frontend services communicate with the backend, and exercise metrics are saved to Firestore with proper error handling.

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC67	FR12	Start the PoseBackend with mocked values.	PoseBackend init values work as intended successfully and `is_calibrating` is set to True.	All assertions pass.	Pass
TC68	FR8	Call `reset()` method on PoseBackend.	Previous analyzer is replaced with a newer analyzer object. New one has `is_calibrating` set to True.	All assertions pass.	Pass
TC69	FR8	Send POST request to `/reset` endpoint.	Response status 200 with JSON {`"ok": true`}. Backend reset method is called.	All assertions pass.	Pass
TC70	FR7, FR26	Send POST request to `/process_frame` with empty image data.	Response status 400 with error message `"Missing imageBase64"`. Landmarks is null.	All assertions pass.	Pass
TC71	FR7, FR26	Send POST request to `/process_frame` with invalid base64 image string.	Response status 400 with error message. No frame processing occurs.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC72	FR7, FR12	Send POST request to `/process_frame` with valid base64 image, `side="RIGHT"`, `mirrored=true`, `legsOnly=true`. Landmarks detected.	Response status 200. Contains all landmarks (33). Metrics contain correct values (angle=45.6, rep_count=5, rep_state="Extension").	All assertions pass.	Pass
TC73	FR7, FR26	Send POST request to `/process_frame` with valid image but no pose landmarks detected.	Response status 200. Landmarks is null. Default metrics returned with `calibrating=True`.	All assertions pass.	Pass
TC74	FR7	Send POST request to `/process_frame` with `mirrored=True` and verify frame flipping.	Frame is flipped horizontally before processing. `cv2.flip` is called once with the frame.	All assertions pass.	Pass
TC75	FR7	Send POST request to `/process_frame` with `mirrored=False` and verify no frame flipping.	Frame is not flipped. `cv2.flip` is not called.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC76	FR7	Test `b64_to_bgr_image` helper function with a valid test image.	Function correctly decodes base64 string back to a numpy array with expected dimensions (100, 100, 3).	All assertions pass.	Pass
TC77	FR12	Send POST requests with `side="RIGHT"` and `side="LEFT"` parameters.	`knee_angle_from_result` is called twice with correct side parameters: first with 'RIGHT', then with 'LEFT'.	All assertions pass.	Pass
TC78	FR7, FR26	Send POST request with valid image but simulate decoding error by returning None.	Response status 400 with error message in JSON.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC79	FR7	Call `processFrame()` with front camera facing, RIGHT side, and a valid base64 image. Mock successful API response.	Function calls axios.post with correct URL, payload includes `mirrored: true`, `side: "RIGHT"`, `legsOnly: true`. Returns the mocked response data with landmarks, connections, and metrics.	All assertions pass.	Pass
TC80	FR7	Call `processFrame()` with back camera facing, LEFT side.	Function calls axios.post with `mirrored: false` in the payload.	All assertions pass.	Pass
TC81	FR7	Call `processFrame()` and simulate a network error.	Function throws an error with message "Network error".	All assertions pass.	Pass
TC82	FR8	Call `resetBackend()` and mock successful response.	Function calls axios.post to `/reset` endpoint with timeout 8000ms. Returns successfully.	All assertions pass.	Pass
TC83	FR8	Call `resetBackend()` and simulate a failure.	Function throws an error with message "Reset failed".	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC84	FR1, FR1.1	Call `saveMetrics()` when no user is logged in (getAuth returns null).	Function throws error "User must be logged in to save metrics".	All assertions pass.	Pass
TC85	FR1.1, FR21	Call `saveMetrics()` when user is logged in but user document does not exist in Firestore.	Function throws error "User data not found in Firestore".	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC86	FR21, FR11	Call `saveMetrics()` with valid metrics data and mocked Firebase responses.	Returns document ID "mock-doc-id". Verifies Firestore calls: doc, getDoc, collection, addDoc. Saved data contains correct fields including actid with timestamp, analysis="pending", exercise="Knee Extension", completed_reps=8, max_height=70.5, min_height=30.1, duration=1, user info, current_angle=45.6, rom_degree=40.4, rep_state="Extension", avg_rep_duration=1.2, current_rep_duration=0.5.	All assertions pass.	Pass
TC87	FR11, FR21	Call `saveMetrics()` with minimal metrics data (missing optional avg_rep_duration and current_rep_duration).	Saved data has avg_rep_duration=0, current_rep_duration=0, duration=0.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC88	FR1.1, FR21	Call `saveMetrics()` when user document exists but has no name field.	Saved data uses "Unknown User" for the name field.	All assertions pass.	Pass
TC89	FR1	Call `saveSessionData()` when no user is logged in.	Function throws error "User must be logged in".	All assertions pass.	Pass
TC90	FR21, FR17, FR11	Call `saveSessionData()` with valid session data.	Returns document ID "mock-doc-id". Saved data contains actid with "session_" prefix, analysis="completed", completed_reps=8, duration=1, exercise="Knee Extension Session", max_height=70.5, min_height=30.1, average_rom=40.4, total_reps=8, avg_rep_duration=1.2.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC91	FR11, FR21	Call `saveSessionData()` without avgRepDuration (uses session duration instead).	Saved data has duration calculated from startTime and endTime (0 seconds), and avg_rep_duration is undefined.	All assertions pass.	Pass
TC92	FR1.1, FR21	Call `saveSessionData()` when user document does not exist.	Saved data uses "Unknown User" for the name field.	All assertions pass.	Pass
TC93	FR1.1, FR21	Call `getUserDisplayName()` with logged in user and existing user document with name field.	Returns "Test User". Verifies doc and getDoc calls with correct parameters.	All assertions pass.	Pass
TC94	FR1.1, FR21	Call `getUserDisplayName()` when user document exists but has no name field.	Returns user email "test@example.com".	All assertions pass.	Pass
TC95	FR1	Call `getUserDisplayName()` when no user is logged in.	Returns "Unknown User".	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC96	FR1.1	Call `getUserDisplayName()` and simulate a Firestore error.	Returns "Unknown User" (error caught and handled gracefully).	All assertions pass.	Pass

Table 7: Performance and Latency Module Test Cases

7 Changes Due to Testing

There were a few changes that were implemented as a result of testing. Note that some of these changes are in progress.

7.1 Usability Testing Results

Analyses from usability testing revealed important user experience insights which has provided clarity with what should be the focus for further development. A subset of recommendations will be developed further, while others will be tabled as out of scope due to project timeline constraints. There were a total of 5 participants in the study, with 2 being PTs, and the rest were those who satisfied the pool criteria. The diverse perspectives proved to be essential by honing in on elements that may provide substantial added value to the application. Both user flows (PT and patient) were tested with the corresponding participants.

The physiotherapist participants had 2 recommendations: A more comprehensive exercise activity record (eg. adding sets in addition to reps) for a clearer picture on the patient's performance, and inclusion of a post-exercise feedback form to provide an insight into performance. (eg. modifying the rep count due to patient's increased pain).

The other participants highlighted the need for audio cues while performing the exercise. Reading the instructions from the acceptable distance necessary for proper application function was troublesome. Secondly, additional

data on the Progress tab would be beneficial for patients to keep track, in a non-technical context, of their recovery.
See Usability Report for additional information.
Note: The usability testing has been completed, but the document's Results and Discussion still remain.

8 Automated Testing

In terms of automated testing, all of the tests for the TypeScript frontend modules as well as the Python backend are automated. When it comes to running the tests, the Python tests are run using `Pytest` by simply typing `pytest` in the command line in the correct root project directory. These tests validate core backend functionality such as the metric calculations and helper functions utilized during exercise processing. Additionally, all of the TypeScript tests can be run in the same manner, however, they run with `Jest` and through running the following command: `npm run test` instead. Ultimately, these tests focus on validating frontend logic including authentication and user account management, user activity tracking, exercise data handling, performance statistics generation, and the feedback module that displays live feedback to users and allows both the physiotherapistss and patients the opportunity to leave comments regarding the user's respective session. It is important to note that the both the Firebase calls and camera components are mocked during testing so that we are able to separately test the logic functionality. Also, the output results for both the frontend and backend automated test suites are printed to the console, ultimately allowing us to be able to quickly identify both the passing and failing tests immediately.

9 Trace to Requirements

The following section showcases the mapping between the various tests performed to the specific requirements that they validate, ultimately displaying the traceability to requirements.

Table 8: Functional Requirements and Corresponding Test Cases

Test Cases	**Functional Requirement(s)**
Account Creation Module Tests (TC1–TC3)	FR1.1, FR1.2
Patient Registration Tests (TC4–TC8)	FR1.1, FR1.3, FR2, FR3
User Authentication Tests (TC9–TC11)	FR1, FR1.1
User Account Tests	
User Management Functions (TC13–TC16)	FR2, FR4, FR6, FR21, FR22, FR23
User Query Function (TC17–TC18)	FR18, FR21, FR25
Specialized functions (TC19–TC21)	FR2, FR4, FR22, SR-AR1, SR-AR2, SR-PR1
Validation helpers (TC22)	FR1, SR-AR1

Table 8 showcases both the functional requirements and their respective test cases.

Table 9: Usability Tests and Corresponding Requirements

Test ID	**Non-Functional Requirement**
UT-1	LFR-AP1
UT-2	UHR-EOU2, UHR-LRN1
UT-3	UHR-EOU1, LFR-ST1

Table 9 maps each of the Usability tests to their respective non-functional requirements.

Table 10: Performance Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
PR-1	PR-SL1, PR-SL2
PR-2	PR-SL1

Table 10 maps each of the Performance tests to their respective non-functional requirements.

Table 11: Reliability & Fault-Tolerance Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
RFT-1	PR-RFT1
RFT-2	PR-RFT2

Table 11 maps each of the Reliability and Fault-Tolerance tests to their respective non-functional requirements.

Table 12: Security Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
SR-1	SR-AR1
SR-2	SR-AR1

Table 12 maps each of the Security tests to their respective non-functional requirements.

Table 13: Maintainability & Support Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
MS-1	OER-WE1, OER-PR1

Table 13 maps each of the Maintainability and Support tests to their respective non-functional requirements.

10 Trace to Modules

The following section displays the traceability of each of the test cases to the specific modules they validated, which ultimately showcases the overall verification of our system.

Table 14: Tests for all of the Modules

Test ID	**Module**
TC1–TC8	M5 (Account Creation)
TC9–TC12	M6 (Valid User)
TC13–TC19	M1 (User Interface), M2 (Profile Activity), M5 (Account Creation), M7 (User Account), M8 (User Activity), M10 (Performance Statistics Generator), M13 (Feedback)
TC20–TC22	M1 (User Interface), M7 (User Account)
UT-1, UT-2, UT-3, PR-1, PR-2, RFT-1, RFT-2	M1 (User Interface)
SR-1, SR-2	M5 (Account Creation), M6 (Valid User)
MS-1, MS-2	M1 (User Interface), M5 (Account Creation), M6 (Valid User)
TC1–TC8	M5 (Account Creation Module)
TC9–TC12	M6 (Valid User Module)
TC13–TC19	M1 (User Interface), M2 (Profile Activity), M5 (Account Creation), M7 (User Account Module), M8 (User Activity), M10 (Performance Statistics Generator), M13 (Feedback)
TC20–TC22	M1 (User Interface), M7 (User Account Module)
TC23–TC30	M10 (Angle Analysis), M11 (Feedback Generation), M12 (Reporting), M13 (Dashboard)
UT-1, UT-2, UT-3, PR-1, PR-2, RFT-1, RFT-2	M1 (User Interface Module)
SR-1, SR-2	M5 (Account Creation Module), M6 (Valid User Module)
MS-1, MS-2	M1 (User Interface Module), M5 (Account Creation Module), M6 (Valid User Module)

Table 14 maps each of the test cases to their respective modules.

11 Code Coverage Metrics

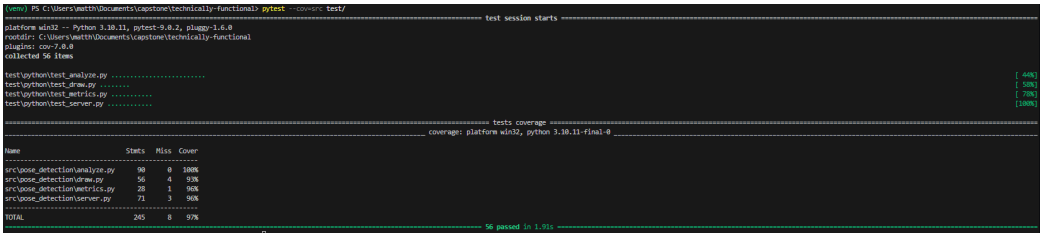

Figure 1: Python code coverage overview showing 97% total coverage across all modules.

Name	Stmts	Miss	Cover
src\pose_detection\analyze.py	90	0	100%
src\pose_detection\draw.py	56	4	93%
src\pose_detection\metrics.py	28	1	96%
src\pose_detection\server.py	71	3	96%
TOTAL	245	8	97%

Figure 2: Detailed Python code coverage by module showing analyze.py (100%), draw.py (93%), metrics.py (96%), and server.py (96%).

The backend code has a total coverage of 97%, with all modules having coverage above 90%. The analyze.py module has 100% coverage, while draw.py, metrics.py, and server.py have 93%, 96%, and 96% coverage respectively. This indicates that the majority of the backend code is well-tested, with only a small portion of code not covered by tests.

Some areas of improvement would be to fully test each backend module to reach 100% coverage. This would be included in future testing plans but the current coverage points to the reliability of the backend code.

12 Extras

[Summary of the status of your extras. If feasible, the extras should be complete by the time of writing the VnV report. If the extras are not complete, you should summarize the plans for completing them. —SS]

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

5. Reflect on your use of CI for continuous integration testing (regression testing). Did your CI work to gatekeep your PRs? Would it have been helpful if you got your CI working earlier in the project? What are your plans to use CI for testing in future projects?

Maham

1. Our team worked really well together. Everyone completed their assigned tasks on time and were available to assist others if needed. We were all communicative, cordial and on schedule.

2. At the start, we split off into completing our 'Extras'. I was responsible for the Usability Testing and I had to ensure that there were results to work with. The team members conducted the tests on participants, while I worked on analysis as the results came in. Nearer to the deadline, we realized that the due date for Extras was moved to much later, (I don't think it was in the calendar until very recently), after which all the members hopped on deck to complete their corresponding module tests. Note to self: please double check your dates.

3. The usability testing results were from our peers and our supervisors. They provided some very useful suggestions and some of those ideas are fairly straightforward to implement.

4. CI proved to be very helpful throughout. Because of continuous regression tests being performed, it was ensured that

5. Our VnV report deviated a fair bit from the orginal plan. Writing the code resulted in a more detailed plan being required. In addition, our team further divided some of the modules, which also resulted in more detailed tests as well. I think I would be able to anticipate them after I have gained sufficient experience and done it a few times. Some of the basic components (eg. Account creation) will have similar implementations everywhere, and those will be easy to anticipate.

6. CI proved to be very helpful throughout. Because of continuos regression tests being performed, it was ensured that any new changes will not break the code. We did use the CI element somewhat earlier, ended up realizing that there were frequent conflicts after which it was decided that we will make more PRs regularly to minimize conflicts. I really like CI, and I see myself using it in the future a fair bit. I will have to concurrently work with others on the same project and it will not become too daunting if and when changes are needed to be made/resolve issues

Vaisnavi:

- *What went well while writing this deliverable?*

  One thing that went well while writing this deliverable was our ability to collaborate and efficiently split up the tasks and sections of this report. Specifically, in the case of the unit tests and traceability, we decided as a group for each member to derive the unit tests and traceability based on the sections of the app that they had worked on. This aided efficiency and productivity overall, and allowed the report to come together easily.

- *What pain points did you experience during this deliverable, and how did you resolve them?*

  One of the biggest pain points I experienced during this deliverable was actually writing the unit tests. Writing the unit tests was actually harder than I had expected because mocking the Firebase database was something I had never done before. Since our authentication and valid user logic uses the Firebase database, we had to mock those database calls instead of using the real database, which took some time to learn and figure out. There was definitely a learning curve, but after looking through some examples and testing different approaches, I was able to get the mocks working correctly. All in all, this pain point allowed me to be able to understand how to test code that depends on external services.

Eman

1. One thing that went well while writing this deliverable was the team's ability to divide the work clearly based on ownership. Since each member had worked on specific parts of the application, it was natural to have each person derive the tests and evaluations for their respective sections. This made the process more efficient and ensured that the tests were written with a solid understanding of the underlying implementation. Communication within the team was consistent throughout, which helped us stay on track and address any blockers quickly.

2. One of the main pain points I experienced was setting up and debugging the network connection between the mobile device and the Flask

backend during testing. The pose detection and recording functionality depended on the phone being able to reach the server, and resolving the IP configuration and network routing issues took considerable time. Once the connection was established, the full recording flow was tested with the real camera and backend, and the system performed as expected.

Matthew:

- *What went well while writing this deliverable?*

  One thing that went well while working on this deliverable was our ability to communicate with each other and work together. We were able to make each meeting meaningful by updating the group on what we were working on, if we were stuck and also future deadlines. This allowed us to stay on track and stay on top of the deadlines. We also divided things based on our module implementation and that helped us clearly define the test case implementation and workload for the other documents.

- *What pain points did you experience during this deliverable, and how did you resolve them?*

  A big pain point was working on the units test. I have not had much experience with testing and it has been a while since I wrote test cases for school. On top of that the difference in technology that we used academically to now was vastly different so I had to learn how to Mock certain objects and functionalities instead of using an actual database or object. I resolved them by just trying different things and looking at examples online. For example, I had to do a lot of reading on Mock and Patches for testing and the role that they play in testing. In the end I was able to create the unit test and learn a bit on how to test code.

Group

- *Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?*

Parts of this document stemmed from speaking to our clients, specifically by receiving feedback from our peers who acted as proxies for physiotherapy patients. When demonstrating our application, they interacted with the system and gave relevant, useful feedback on the usability of the interface as well as how clear and easy to grasp the real-time exercise feedback was. This mainly had an impact on the sections related to usability testing, interface design, and how feedback is presented to users, especially the requirement for clear instructions and visible/audio cues during exercises. Moreover, the other parts of the document, such as functional requirement evaluation and technical implementation, did not come directly from speaking to our clients. These were based on the requirements defined in our SRS and the testing done by our core team, since these parts relate more to the development logic and system reliability rather than user interaction.

- *In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)*

  Our VnV Plan deviated from the activities that were actually conducted in a few notable ways. The original plan outlined testing at a higher level of abstraction, targeting broader system functionalities rather than individual module functions. As development progressed and the codebase became more detailed, it became clear that more granular unit tests were necessary. This also coincided with our team further subdividing some of the modules during implementation, which resulted in more detailed tests being required. This led to the addition of the User Account module test suite, which did not exist in the original plan at all. As this module grew in scope and became central to both patient and PT workflows, dedicated testing for functions such

as `getUserData()`, `updateUser()`, and `deletePTAccount()` became essential. The usability testing itself proceeded largely as planned, though the participant pool was smaller than initially intended due to scheduling constraints. The structured questionnaire format, pre-session, session, and post-session, was maintained as designed and provided consistent data across all five participants.

- *Reflect on your use of CI for continuous integration testing (regression testing). Did your CI work to gatekeep your PRs? Would it have been helpful if you got your CI working earlier in the project? What are your plans to use CI for testing in future projects?*

  Our project currently uses GitHub Actions for CI, but as of right now, the workflow is limited and mainly displays the pull request activity and build checks. With our implemented unit tests for both the frontend and backend, we want to integrate these directly into the CI pipeline. Specifically, the GitHub Actions workflow could be configured to automatically run the Jest test suite for the TypeScript frontend and the Pytest test suite for the Python backend whenever a pull request is opened or updated. In this case, if any test fails, the workflow would essentially fail, which would ultimately prevent changes from being merged until the issue is resolved. With this proposed change, this would make the CI pipeline much more effective for regression testing, as it would automatically verify that new code trying to be merged in does not break existing functionality. It is important to note that it would have been helpful to set this up earlier in the project, and as a result, for future projects, we would aim to do so. Thus, overall for future projects, we would plan to set up CI earlier and integrate our automated test suites (such as Jest and Pytest) so that the tests run automatically on every pull request and prevent merges if any tests fail.